EX No:06

Date:

Model Building using Classification Algorithm for Weekly Dataset

6. In this problem, you will develop a model to predict whether a given car gets high or low gas mileage based on the Auto data set.

LOAD LIBRARY:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
summarize)
from ISLP import confusion_table
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix
```

LOAD DATA:

```
data = load_data('Auto')
data.head()
```

(a) Create a binary variable, mpg01, that contains a 1 if mpg contains a value above its median, and a 0 if mpg contains a value below its median. You can compute the median using the median() method of the data frame. Note you may find it helpful to add a column mpg01 to the data frame by assignment. Assuming you have stored the data frame as Auto, this can be done as follows: Auto['mpg01'] = mpg01

CODE:

```
median_mpg = data["mpg"].median()

print(median_mpg)
```

OUTPUT:

```
22.75
```

CODE:

```
data["mpg01"] = np.where(
data["mpg"] > median_mpg,
   1, 0)
```

OUTPUT:

name	mpg	mpg01
chevrolet chevelle malibu	18.0	0
buick skylark 320	15.0	0
plymouth satellite	18.0	0
amc rebel sst	16.0	0
ford torino	17.0	0
ford galaxie 500	15.0	0
chevrolet impala	14.0	0
plymouth fury iii	14.0	0
pontiac catalina	14.0	0
amc ambassador dpl	15.0	0

(b) Explore the data graphically in order to investigate the association between mpg01 and the other features. Which of the other features seem most likely to be useful in predicting mpg01? Scatterplots and boxplots may be useful tools to answer this question. Describe your findings.

CODE:

```
sns.boxplot(data=data, x="mpg01", y="cylinders")
plt.title("Cylinders vs mpg01")
plt.show()

sns.boxplot(data=data, x="mpg01", y="displacement")
plt.title("Displacement vs mpg01")
plt.show()

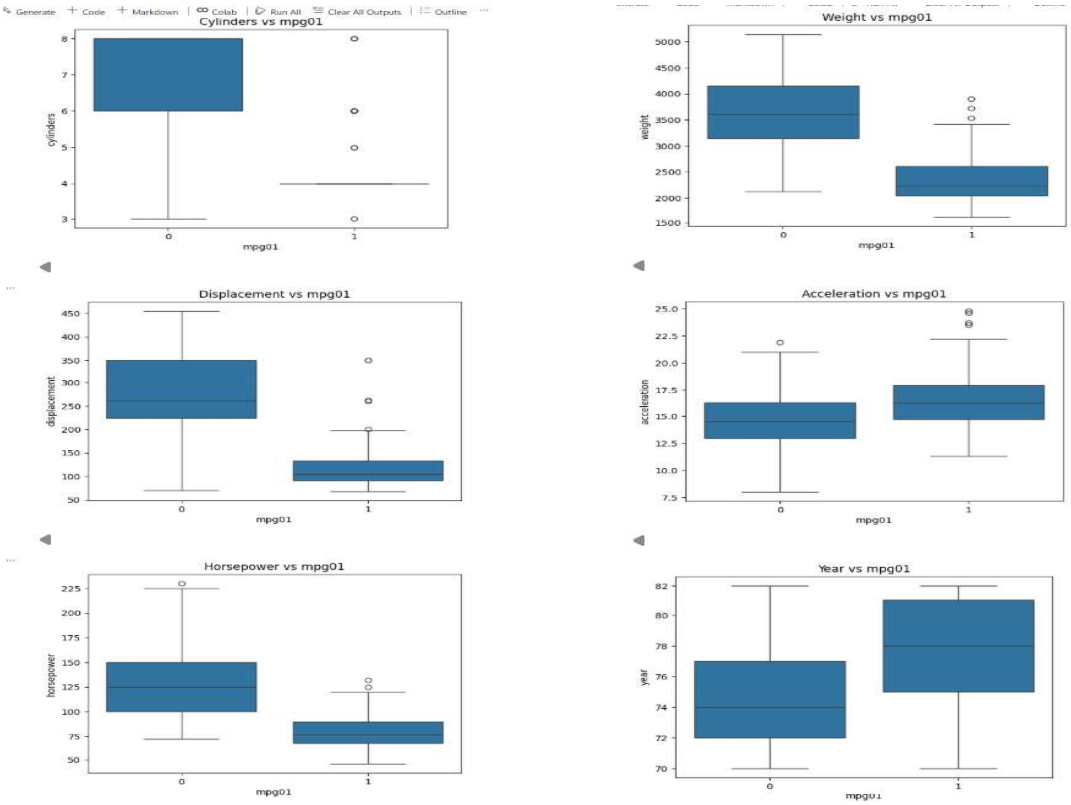
sns.boxplot(data=data, x="mpg01", y="horsepower")
plt.title("Horsepower vs mpg01")
plt.show()

sns.boxplot(data=data, x="mpg01", y="weight")
plt.title("Weight vs mpg01")
plt.show()

sns.boxplot(data=data, x="mpg01", y="acceleration")
plt.title("Acceleration vs mpg01")
plt.show()

sns.boxplot(data=data, x="mpg01", y="year")
plt.title("Year vs mpg01")
plt.show()

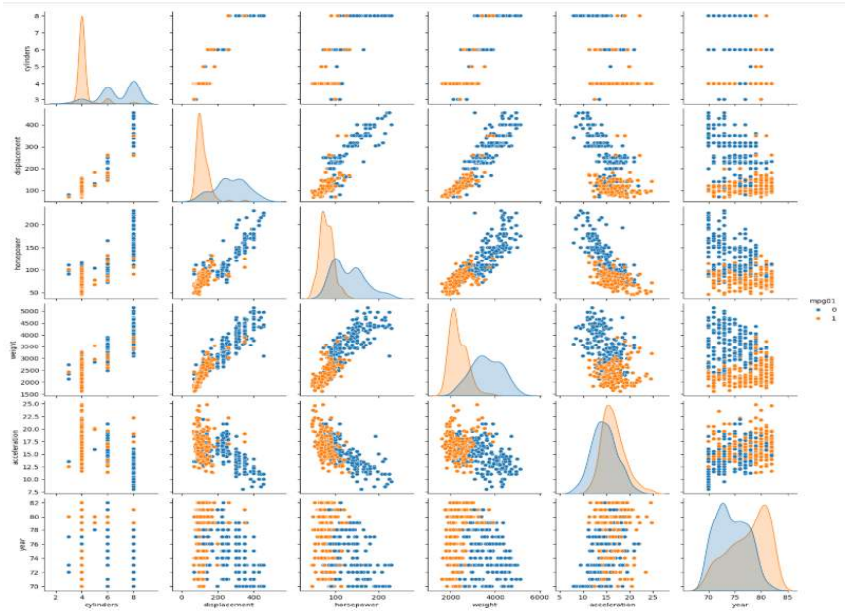
sns.boxplot(data=data, x="mpg01", y="origin")
plt.title("Origin vs mpg01")
plt.show()
```



CODE:

```
sns.pairplot(data,vars=['cylinders','displacement','horsepower','weight','acceleration','year'],hue='
mpg01')
plt.show()
```

OUTPUT:



(c) Split the data into a training set and a test set.

CODE:

```
X = data[["cylinders", "displacement", "horsepower", "weight", "acceleration", "year"]]
y = data[["mpg01"]]
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.30, random_state=123,
    stratify=y)
print(X_train.shape)
print(X_test.shape)
```

(d) Perform LDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE:

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
```

OUTPUT:



CODE:

```
lda_pred = lda.predict(X_test)
confusion_table(y_test, lda_pred)
```

OUTPUT:

Truth	0	1
Predicted		
0	48	11
1	1	58

CODE:

```
lda_accuracy = accuracy_score(y_test, lda_pred)
print("LDA Accuracy:", lda_accuracy)
```

OUTPUT:

```
LDA Accuracy: 0.8983050847457628
```

CODE:

```
lda_error = 1 - lda_accuracy
print("LDA Test Error:", lda_error)
```

OUTPUT:

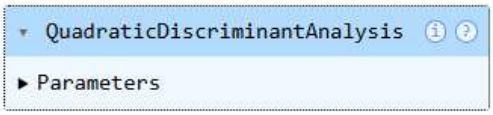
```
LDA Test Error: 0.10169491525423724
```

(e) Perform QDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE:

```
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
```

OUTPUT:



CODE:

```
qda_pred = qda.predict(X_test)
confusion_table(y_test, qda_pred)
```

OUTPUT:

Truth	0	1
Predicted		
0	51	8
1	3	56

CODE:

```
qda_accuracy = accuracy_score(y_test, qda_pred)
print("QDA Accuracy:", qda_accuracy)
```

OUTPUT:

```
QDA Accuracy: 0.9067796610169492
```

CODE:

```
qda_error = 1 - qda_accuracy
print("QDA Test Error:", qda_error)
```

OUTPUT:

```
QDA Test Error: 0.09322033898305082
```

(e) Perform logistic regression on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE:

```
X = data[['cylinders','displacement','horsepower','weight','year']]
y = data['mpg01']
x_train = MS(['cylinders','displacement','horsepower','weight','year']).fit_transform(X_train)
model = sm.GLM(y_train,x_train,family=sm.families.Binomial()).fit()
print(model.summary())
```

OUTPUT:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                   mpg01   No. Observations:                  274
Model:                             GLM   Df Residuals:                      268
Model Family:                Binomial   Df Model:                            5
Link Function:                   Logit   Scale:                          1.0000
Method:                           IRLS   Log-Likelihood:                -54.306
Date:                 Tue, 29 Sep 2026   Deviance:                       108.61
Time:                         07:03:10   Pearson chi2:                     180.
No. Iterations:                      8   Pseudo R-squ. (CS):             0.6284
Covariance Type:             nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept    -15.7291      5.989     -2.626      0.009     -27.467      -3.991
cylinders     -0.0170      0.459     -0.037      0.971      -0.917       0.883
displacement   0.0054      0.012      0.460      0.645      -0.018       0.028
horsepower    -0.0348      0.018     -1.951      0.051      -0.070       0.000
weight        -0.0054      0.001     -4.300      0.000      -0.008      -0.003
year           0.4409      0.095      4.635      0.000       0.254       0.627
==============================================================================
```

(g) Perform KNN on the training data, with several values of K, in order to predict mpg01. Use only the variables that seemed most associated with mpg01 in (b). What test errors do you obtain? Which value of K seems to perform the best on this data set?

LOAD LIBRARY:

```
from sklearn.neighbors import KNeighborsClassifier as KNC
from sklearn.preprocessing import StandardScaler
```

CODE:

```
knn1 = KNC(n_neighbors=1)
knn1.fit(X_train, y_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, y_test)
```

OUTPUT:

Truth	0	1
Predicted		
0	44	3
1	15	56

CODE:

```
error1 = 1 - accuracy_score(y_test, knn1_pred)
print("K = 1 Test Error:", error1)
```

OUTPUT:

```
K = 1 Test Error: 0.15254237288135597
```

CODE:

```
knn3 = KNC(n_neighbors=1)
knn3.fit(X  train, y  train)
knn3_pred = knn3.predict(X_test)
confusion_table(knn3_pred, y_test)
```

OUTPUT:

Truth	0	1
Predicted		
0	44	3
1	15	56

CODE:

```
error1 = 1 - accuracy_score(y_test, knn3_pred)
print("K = 3 Test Error:", error1)
```

OUTPUT:

```
K = 3 Test Error: 0.15254237288135597
```

CODE:

```
knn5 = KNC(n_neighbors=1)
knn5.fit(X_train, y_train)
knn5_pred = knn5.predict(X_test)
confusion_table(knn5_pred, y_test)
```

OUTPUT:

Truth	0	1
Predicted		
0	44	3
1	15	56

CODE:

```
error1 = 1 - accuracy_score(y_test, knn5_pred)
print("K = 5 Test Error:", error1)
```

OUTPUT:

```
K = 5 Test Error: 0.15254237288135597
```

INTERPRETATION:

- (a) The median mpg is 22.75, so mpg01 = 1 marks cars with above-median mileage and 0 the rest, which gives two roughly balanced classes.
- (b) The boxplots show high-mpg cars (mpg01 = 1) are much lighter (median weight about 2,200 lb against about 3,600 lb), have smaller engines (median displacement about 105 against about 260), lower horsepower (about 78 against about 125) and are mostly 4-cylinder. They are also newer (median year about 78 against 74). Acceleration and origin separate the classes less clearly. Cylinders, displacement, horsepower, weight and year are therefore the most useful predictors of mpg01.
- (c) The data was split 70/30 (stratified): 274 training and 118 test observations.
- (d) LDA gives a test accuracy of 89.8% (test error 10.2%, 12 of 118 cars misclassified).
- (e) QDA gives a test accuracy of 90.7% (test error 9.3%, 11 of 118 misclassified), only one car better than LDA, so the two are effectively equal and the class boundary is close to linear.
- (f) In the logistic regression (274 training cars), weight (-0.0054, $p < 0.001$) and year (0.4409, $p < 0.001$) are significant, horsepower is borderline ($p = 0.051$), and cylinders and displacement are not significant because they overlap with weight. The pseudo R-squared is 0.628. The record prints the model summary but does not compute a test error for it.
- (g) KNN gives a test error of 15.25% (18 of 118 misclassified) for K = 1, 3 and 5. The identical values arise because the K = 3 and K = 5 cells in the record still call KNC(n_neighbors=1); they need to be changed to 3 and 5 before the values of K can be compared. The features were also not standardised, so weight and displacement dominate the distance.
- Overall, LDA and QDA (about 9 to 10% test error) outperform KNN (about 15%) on this data, and roughly 9 out of 10 cars are correctly classified as high or low mileage.